

WEEK 4

Scenario :

The Order Management System is used to store and manage customer order information and the products included in each order. It contains two tables: **ORDER_ENTITY** and **ORDER_DETAILS_ENTITY**.

ORDER_ENTITY :

The ORDER_ENTITY table stores the main information about each customer order.

Attributes:

- **ORDER_ID** – Unique ID for each order
- **CUSTOMER_ID** – Identifies the customer
- **ORDER_DATE** – Date of the order
- **TOTAL_AMOUNT** – Total amount of the order
- **ORDER_STATUS** – Current status of the order

```
SQL> CREATE TABLE ORDER_ENTITY (  
2     ORDER_ID INT,  
3     CUSTOMER_ID INT,  
4     ORDER_DATE DATE,  
5     TOTAL_AMOUNT NUMBER,  
6     ORDER_STATUS VARCHAR2(50)  
7 );
```

Table created.

ORDER_DETAILS_ENTITY:

The ORDER_DETAILS_ENTITY table stores the details of the products included in each order.

Attributes:

- **ORDER_DETAIL_ID** – Unique ID for each order detail
- **ORDER_ID** – Identifies the related order
- **PRODUCT_ID** – Identifies the ordered product
- **QUANTITY** – Number of products ordered
- **PRICE** – Price of the product/order detail

```
SQL> CREATE TABLE ORDER_DETAILS_ENTITY (  
2     ORDER_DETAIL_ID INT,  
3     ORDER_ID INT,  
4     PRODUCT_ID INT,  
5     QUANTITY VARCHAR2(50),  
6     PRICE INT  
7 );
```

Table created.

Establish Relationships and Constraints :

One Order can have many Order Details.

The relationship between **ORDER_ENTITY** and **ORDER_DETAILS_ENTITY** is **One to Many (1 → *)**.

PRIMARY KEY:

The following Primary Keys are added:

- **ORDER_ID** – Primary Key of **ORDER_ENTITY** table.
- **ORDER_DETAIL_ID** – Primary Key of **ORDER_DETAILS_ENTITY** table.

A Primary Key uniquely identifies each record in a table.

```
SQL> ALTER TABLE ORDER_ENTITY
2 ADD CONSTRAINT PK_ORDER
3 PRIMARY KEY (ORDER_ID);
```

Table altered.

```
SQL> ALTER TABLE ORDER_DETAILS_ENTITY
2 ADD CONSTRAINT PK_ORDER_DETAILS
3 PRIMARY KEY (ORDER_DETAIL_ID);
```

Table altered.

FOREIGN KEY:

The following Foreign Key is used in the **ORDER_DETAILS_ENTITY** table:

- **ORDER_ID** references **ORDER_ENTITY(ORDER_ID)**

This Foreign Key establishes the relationship between the two tables.

```
SQL> ALTER TABLE ORDER_DETAILS_ENTITY
2 ADD CONSTRAINT FK_ORDER
3 FOREIGN KEY (ORDER_ID)
4 REFERENCES ORDER_ENTITY(ORDER_ID);
```

Table altered.

```
SQL> ALTER TABLE ORDER_DETAILS_ENTITY
  2  ADD CONSTRAINT FK_ORDER
  3  FOREIGN KEY (ORDER_ID)
  4  REFERENCES ORDER_ENTITY(ORDER_ID);
```

Table altered.

NOT NULL Constraint:

NOT NULL ensures that a column cannot contain an empty (NULL) value.

```
SQL> ALTER TABLE ORDER_DETAILS_ENTITY
  2  MODIFY PRODUCT_ID NOT NULL;
```

Table altered.

```
SQL> ALTER TABLE ORDER_DETAILS_ENTITY
  2  MODIFY QUANTITY NOT NULL;
```

Table altered.

```
SQL> ALTER TABLE ORDER_DETAILS_ENTITY
  2  MODIFY PRICE NOT NULL;
```

Table altered.

CHECK Constraint:

CHECK validates that the values in a column satisfy a specified condition.

- TOTAL_AMOUNT > 0 – Ensures the total order amount is positive.
- QUANTITY > 0 – Ensures the ordered quantity is positive.
- PRICE > 0 – Ensures the product price is positive.

```
SQL> ALTER TABLE ORDER_ENTITY
  2  ADD CONSTRAINT CHK_ORDER_AMOUNT
  3  CHECK (TOTAL_AMOUNT > 0);
```

Table altered.

```
SQL> ALTER TABLE ORDER_DETAILS_ENTITY
  2  ADD CONSTRAINT CHK_QUANTITY
  3  CHECK (QUANTITY > 0);
```

Table altered.

```
SQL> ALTER TABLE ORDER_DETAILS_ENTITY
 2  ADD CONSTRAINT CHK_ORDER_PRICE
 3  CHECK (PRICE > 0);

Table altered.
```

UNIQUE Constraint:

UNIQUE ensures that duplicate values are not allowed in a column.:

```
SQL> ALTER TABLE ORDER_ENTITY
 2  ADD CONSTRAINT UQ_CUSTOMER_ORDER
 3  UNIQUE (CUSTOMER_ID);

Table altered.
```

Insert Values to ORDER_ENTITY Table:

```
SQL> INSERT INTO ORDER_ENTITY
 2  VALUES (1001, 201, TO_DATE('20-09-2026','DD-MM-YYYY'), 1500, 'CONFIRMED');

1 row created.

SQL>
SQL> INSERT INTO ORDER_ENTITY
 2  VALUES (1002, 202, TO_DATE('21-09-2026','DD-MM-YYYY'), 2200, 'PENDING');

1 row created.

SQL>
SQL> INSERT INTO ORDER_ENTITY
 2  VALUES (1003, 203, TO_DATE('22-09-2026','DD-MM-YYYY'), 1800, 'SHIPPED');

1 row created.

SQL>
SQL> INSERT INTO ORDER_ENTITY
 2  VALUES (1004, 204, TO_DATE('23-09-2026','DD-MM-YYYY'), 2500, 'DELIVERED');

1 row created.

SQL>
SQL> INSERT INTO ORDER_ENTITY
 2  VALUES (1005, 205, TO_DATE('24-09-2026','DD-MM-YYYY'), 1200, 'CONFIRMED');

1 row created.
```

```
SQL> SELECT * FROM ORDER_ENTITY;
```

ORDER_ID	CUSTOMER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
1001	201	20-SEP-26	1500	CONFIRMED
1002	202	21-SEP-26	2200	SHIPPED
1003	203	22-SEP-26	1800	SHIPPED
1004	204	23-SEP-26	2500	DELIVERED
1005	205	24-SEP-26	1200	CONFIRMED
1006	206	25-SEP-26	1700	CONFIRMED
1007	207	26-SEP-26	1900	PENDING
1008	208	27-SEP-26	2300	SHIPPED
1009	209	28-SEP-26	1400	CONFIRMED
1010	210	29-SEP-26	2800	DELIVERED
1011	211	30-SEP-26	1600	CONFIRMED

11 rows selected.

INSERT VALUES TO ORDER DETAIL ENTITY:

```
SQL> INSERT INTO ORDER_DETAILS_ENTITY
2 VALUES (11, 1001, 101, 2, 1200);

1 row created.

SQL>
SQL> INSERT INTO ORDER_DETAILS_ENTITY
2 VALUES (12, 1002, 102, 3, 2100);

1 row created.

SQL>
SQL> INSERT INTO ORDER_DETAILS_ENTITY
2 VALUES (13, 1003, 103, 2, 1800);

1 row created.

SQL>
SQL> INSERT INTO ORDER_DETAILS_ENTITY
2 VALUES (14, 1004, 104, 4, 2600);

1 row created.

SQL>
SQL> INSERT INTO ORDER_DETAILS_ENTITY
2 VALUES (15, 1005, 105, 3, 1200);

1 row created.
```

```
SQL> SELECT * FROM ORDER_DETAILS_ENTITY;
```

ORDER_DETAIL_ID	ORDER_ID	PRODUCT_ID	QUANTITY	PRICE
2	11	1001	101	1200
3	12	1002	102	2100
2	13	1003	103	1800

ORDER_DETAIL_ID	ORDER_ID	PRODUCT_ID	QUANTITY	PRICE
4	14	1004	104	2600
3	15	1005	105	1200
2	16	1006	101	1200

ORDER_DETAIL_ID	ORDER_ID	PRODUCT_ID	QUANTITY	PRICE
2	17	1007	102	1400
2	18	1008	103	1800
2	19	1009	104	1300

ORDER_DETAIL_ID	ORDER_ID	PRODUCT_ID	QUANTITY	PRICE
3	20	1010	105	1200
2	21	1011	101	1200

UPDATE OPERATION :

The order status of **ORDER_ID 1002** was successfully updated from **PENDING** to **SHIPPED**.

```
SQL> UPDATE ORDER_ENTITY
2 SET ORDER_STATUS = 'SHIPPED'
3 WHERE ORDER_ID = 1002;
```

```
1 row updated.
```

DELETE OPERATION:

The order detail record with **ORDER_DETAIL_ID 21** was successfully deleted from the **ORDER_DETAILS_ENTITY** table.

```
SQL> DELETE FROM ORDER_DETAILS_ENTITY
2 WHERE ORDER_DETAIL_ID = 21;
```

```
1 row deleted.
```

The order record with **ORDER_ID_1011** was successfully deleted from the **ORDER_ENTITY** table.

```
SQL> DELETE FROM ORDER_ENTITY
2 WHERE ORDER_ID = 1011;
```

```
1 row deleted.
```

REPORT 1;

Customer Order History: Customer Name , Order ID ,Order Date , Total Amount ,Status.

```
SQL> SELECT CUSTOMER_ID,
2 ORDER_ID,
3 ORDER_DATE,
4 TOTAL_AMOUNT,
5 ORDER_STATUS,
6 FROM ORDER_ENTITY
7 ORDER BY ORDER_ID;
```

CUSTOMER_ID	ORDER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
201	1001	20-SEP-26	1500	CONFIRMED
202	1002	21-SEP-26	2200	SHIPPED
203	1003	22-SEP-26	1800	SHIPPED

CUSTOMER_ID	ORDER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
204	1004	23-SEP-26	2500	DELIVERED
205	1005	24-SEP-26	1200	CONFIRMED
206	1006	25-SEP-26	1700	CONFIRMED

CUSTOMER_ID	ORDER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
207	1007	26-SEP-26	1900	PENDING
208	1008	27-SEP-26	2300	SHIPPED
209	1009	28-SEP-26	1400	CONFIRMED

CUSTOMER_ID	ORDER_ID	ORDER_DAT	TOTAL_AMOUNT	ORDER_STATUS
210	1010	29-SEP-26	2800	DELIVERED

```
10 rows selected.
```

Report 2 ;

Product-wise Order : Number of times each product was ordered, Total quantity sold.

```
SQL> SELECT PRODUCT_ID,
2          COUNT(ORDER_ID) AS NUMBER_OF_TIMES_ORDERED,
3          SUM(QUANTITY) AS TOTAL_QUANTITY_SOLD
4 FROM ORDER_DETAILS_ENTITY
5 GROUP BY PRODUCT_ID
6 ORDER BY PRODUCT_ID;
```

PRODUCT_ID	NUMBER_OF_TIMES_ORDERED	TOTAL_QUANTITY_SOLD
101	2	4
102	2	5
103	2	4
104	2	6
105	2	6

Report 3 ;

Customer Purchase Analysis : Customers with maximum orders ,Total spending by each customer , Average order value

```
SQL> SELECT CUSTOMER_ID,
2          COUNT(ORDER_ID) AS MAXIMUM_ORDERS,
3          SUM(TOTAL_AMOUNT) AS TOTAL_SPENDING,
4          AVG(TOTAL_AMOUNT) AS AVERAGE_ORDER_VALUE
5 FROM ORDER_ENTITY
6 GROUP BY CUSTOMER_ID
7 ORDER BY CUSTOMER_ID;
```

CUSTOMER_ID	MAXIMUM_ORDERS	TOTAL_SPENDING	AVERAGE_ORDER_VALUE
201	1	1500	1500
202	1	2200	2200
203	1	1800	1800
204	1	2500	2500
205	1	1200	1200
206	1	1700	1700
207	1	1900	1900
208	1	2300	2300
209	1	1400	1400
210	1	2800	2800

10 rows selected.